面向情感陪伴场景的数字人对话模型

——数据构建、参数高效微调与评测

2026 动感地带 AI+ 高校创智计划　主题二

参赛队：心屿智创队

2026 年 10 月 18 日

摘要

本文报告了参加 2026 动感地带 AI+ 高校创智计划主题二“数字人综合情感陪伴对话模型”的完整工作。赛题要求模型在给定对话历史的条件下，一次输出包含回复文本、情感标签、用户画像和记忆引用的结构化结果，因此该任务本质上是一个单次前向的多任务生成问题，而非检索或状态维护问题。我们从官方公开数据出发构建了 17710 条监督样本，在 Qwen2.5-1.5B-Instruct 上以 LoRA 进行参数高效微调，并在官方验证集上以官方推理脚本评测。情感标签识别准确率达到 70.67%，约为平凡基线（恒输出多数类）的 2.49 倍；用户画像的整对象精确匹配率为 30.33%，低于“恒输出空数组”所能达到的 55.67%，是本工作未解决的问题。本文详细记录了两处关键的数据处理决策、一次关于训练量的消融实验，以及对画像字段失效机制的分析，以供后续改进参考。

目录

1 任务分析 **3**

1.1 评测接口的实际约束 3

1.2 平凡基线 3

2 数据构建 **3**

2.1 从对话数据到监督样本 3

2.2 截断与清洗 3

2.3 记忆引用字段的处理 4

2.4 提示词一致性与一处官方实现缺陷 4

3 模型与训练 **4**

3.1 基座选择 4

3.2 训练配置 4

3.3 训练过程 5

4 实验与结果 **5**

4.1 评测设置 5

4.2 情感标签 5
4.3 用户画像 5
4.4 回复文本 5
4.5 训练量的消融 5

5 讨论 **6**
5.1 style 字段的失效机制 6
5.2 评分口径对结论的影响 6
5.3 其他限制 6

6 结论与后续工作 **7**

7 复现说明 **7**

1 任务分析

1.1 评测接口的实际约束

赛题提供的推理脚本 `run_inference.py` 定义了唯一的评测入口：读入一个 jsonl 文件，对每一行（一段对话历史）调用一次模型，将输出解析为一个 JSON 对象并写入提交文件。脚本内部固定了系统提示词，明确要求输出四个字段：`response_text`（回复文本）、`emotion_label`（情感标签，取自 16 个英文枚举）、`user_profile`（三组标签数组，分别为人格特质 10 个、兴趣 12 个、表达风格 11 个）以及 `memory_refs`。

这一定义排除了若干看起来合理但实际上不适用的技术路线。系统不维护跨轮状态，因此工作记忆与多轮状态机无用；测试集不提供记忆库，因此检索增强无法作用于该字段；模型每轮只有一次生成机会，因此采样投票、多轮自我修正等推理期技巧也无法在评测中生效。换言之，所有提升都必须体现在单次前向的输出分布上。

1.2 平凡基线

在讨论任何方法之前，需要先确定必须超越的下界。我们在官方验证集 `val_public` 的前 300 个预测点上统计了两个平凡策略的表现：情感标签恒输出出现频次最高的 `anxiety`，可得准确率 0.2833；用户画像恒输出三个空数组，可得整对象精确匹配率 0.5567。后一个数字之所以偏高，是因为参考集中有相当比例的样本本身画像为空，这一现象在后文第五节的讨论中还会用到。

2 数据构建

2.1 从对话数据到监督样本

官方训练集 `train_public.jsonl` 包含 8338 段完整对话、共 18657 个预测点。每个预测点由三部分组成：目标用户轮次编号、对应的助手回复轮次编号，以及该轮次的参考输出。由于参考回复与对话中既有的助手轮次逐字相同（18657 条全部满足），该数据集可直接视为标准的下一轮回复预测任务。

构建监督样本时，输入取截至目标用户轮次的全部对话，输出取参考目标的 JSON 序列化结果。系统提示词与对话渲染均不重新实现，而是直接导入官方脚本中的 `SYSTEM_PROMPT` 常量与 `conversation_text()` 函数，以避免人工转录带来的格式漂移。经验证，训练输入与推理输入在字符级别完全一致。

最终得到 17710 条训练样本与 943 条内部验证样本，二者按对话划分，不存在同一段对话跨集合的情况。

2.2 截断与清洗

原始数据中有一个容易被忽略的细节：`turns` 字段保存的是完整对话，而只有 5617/18657 个预测点位于对话末尾，其余约七成的预测点后面还接着若干轮。如果在构建输入时直接使用完整对话，参考答案就会出现在输入里。因此所有样本都在目标用户轮次处截断，并断言截断后的末轮角色为用户。

此外，assistant_response_turn_id 理论上恒等于 target_user_turn_id 加一，实测 18653/18657 成立，剩余 4 条为负一（助手轮出现在用户轮之前），属于数据噪声，已剔除。

2.3 记忆引用字段的处理

训练集中有 97.7% 的参考目标带有非空的 mem_ 编号。但官方系统提示词明确要求该字段固定输出空数组，理由是测试集不提供公开记忆库，且数据中不存在编号到记忆文本的任何映射关系，无法从对话历史反推。考虑到训练目标必须与推理指令保持一致，我们将该字段统一置为空数组；若保留原值，模型学到的将是一个在推理时既无法复现、又违背系统提示的行为。

代价是该字段的精确匹配率约为 0.02。这属于官方设定导致的结构性不可得分项，而非模型能力问题。

2.4 提示词一致性与一处官方实现缺陷

在核对提示词一致性时发现，官方脚本用", ".join(EMOTIONS) 拼接情感枚举，而 EMOTIONS 是一个 Python set，其迭代顺序受哈希种子随机化影响。连续三次启动进程，得到的枚举顺序各不相同。这意味着官方脚本每次运行生成的系统提示词在枚举顺序上都是不同的。

该缺陷带来两个后果：训练数据无法复现，且训练与推理之间的枚举顺序必然不一致。我们的处理是改用 sorted() 固定顺序，并加入可复现性校验（连续多次构建的数据文件 sha256 完全一致）。这一处理不影响枚举集合本身，因而不改变任务语义。

3 模型与训练

3.1 基座选择

本工作最终采用 Qwen2.5-1.5B-Instruct。这一选择并非出于效果最优的考虑，而是训练期间硬件条件所迫。赛事平台在训练窗口内的西南一区与西南二区，1、2、4、8 卡四档规格均显示售罄，20 卡时配额虽已到账却无法开机；可用算力仅有一台配备 RTX 4060 Laptop（8 GB 显存）的笔记本。在 8 GB 显存下，7B 模型只能以极短序列长度训练，实际不可行，1.5B 是唯一能在完整序列长度上跑通的规模。

3.2 训练配置

训练采用 LoRA，秩 32、缩放系数 64、作用于全部线性层，权重以 bf16 保存。序列长度设为 2048，覆盖约 78% 的样本，其余样本的尾部会被截断。批大小为 1，梯度累积 16 步，等效批大小 16；学习率 1×10^{-4}，余弦衰减，预热比例 5%，共训练 2 个 epoch（2216 步）。训练时显存占用约 7.9 GB。

未采用 4-bit QLoRA 是出于稳定性考虑：1.5B 的 bf16 权重可以完整放入显存，而 bitsandbytes 在 Windows 上的兼容性存在风险，权衡之下选择了实现更简单、故障面更小的方案。

3.3 训练过程

训练损失与验证损失均单调下降，未出现震荡或发散。第 100 步验证损失为 1.6802，第 1000 步降至 1.4552，训练结束（第 2216 步）时为 1.3883。学习率按余弦曲线正常衰减，末段接近零，说明调度正常收尾。

4 实验与结果

4.1 评测设置

评测使用官方验证集 `val_public` 的前 300 个预测点，转换为官方输入格式后直接调用官方 `run_inference.py`（transformers 后端）。评分环节复现了 `normalize_prediction` 的合法化语义：超出枚举范围的取值一律置空，并按空值计为预测失败，以保证与评测方口径一致。

4.2 情感标签

模型在该字段上的准确率为 0.7067，宏平均 F1 为 0.5743，相对 0.2833 的多数类基线提升约 2.49 倍。混淆主要发生在语义相近的类别之间：`gratitude` 被判为 `relaxed` 共 9 次，`shame` 被判为 `anxiety` 4 次，`mixed` 与 `pride` 各被判为 `relaxed` 4 次。这些错误集中在情绪强度或极性接近的类别上，属于边界判断偏差，而非随机错误。

4.3 用户画像

画像字段的表现明显弱于情感字段。三个子字段的精确匹配率分别为 0.6500、0.6400 和 0.5967，逐标签 micro-F1 分别为 0.2800、0.1871 和 0.0000；整对象精确匹配率为 0.3033。

其中 `style` 字段完全失效：模型在 300 条样本中没有任何一条输出非空值，因此该字段的精确率、召回率与 F1 均为零。这一现象需要单独讨论，见第五节。

4.4 回复文本

模型没有产生空回复，长度中位数为 153 字符，与参考的 146 字符接近；字符级 ROUGE-L 约为 0.2265。作为对照，未微调的基座模型完全不遵循输出契约，其输出形如 `{"response": "..."}`,四个必需字段缺失,经合法化后情感与画像字段全部为空。微调后格式正确率达到 100%。

4.5 训练量的消融

为判断训练量是否充分，我们在第 1650 步（1.5 个 epoch）与第 2216 步（2 个 epoch）两个检查点上用同一份 300 条评测集做了对比。

验证损失继续下降，但所有任务指标均未提升，差异落在 300 样本的统计噪声范围内（情感一项相差三条样本）。这一结果提示：验证损失衡量的是整段 JSON 的 token 似然，而其中绝大部分 token 属于长文本回复，损失下降未必反映在四个离散字段上。对于以字段命中率为目标的赛题，仅凭损失曲线判断训练是否充分并不可靠。

表 1 训练量消融结果

指标	1650 步（1.5 epoch）	2216 步（2 epoch）
验证损失	1.3977	1.3883
情感准确率	0.7167	0.7067
情感宏平均 F1	0.5940	0.5743
画像整对象精确匹配	0.3067	0.3033
画像三字段 F1 之和	0.488	0.467

需要说明的是，1.5 epoch 的检查点因配置中 `save_total_limit` 的限制，在保存后续检查点时被自动清理，因此最终提交采用 2 个 epoch 的版本。两版差距既在噪声内，这一取舍不影响结论。

5 讨论

5.1 style 字段的失效机制

`style` 字段的失效不是随机误差，而是一种系统性的输出抑制。模型在仅有的少数非空预测中精确率为 1.0（1.5 epoch 时），说明它具备判断能力，却在绝大多数情况下选择留空。

我们对其成因给出三点推断。其一，参考分布中该字段为空的比例约为 60%，空是最常见的单一答案，在类别不均衡时模型倾向回归多数类。其二，`memory_refs` 在全部训练样本中均为空数组，且位于 JSON 末尾紧邻 `style`，模型可能学到了“尾部字段应为空”的位置性规律。其三，1.5B 规模的模型对这类需要从用户措辞中推断表达风格的任务，判别信号本就较弱。

针对前两点，我们构建了一版干预数据：将 JSON 键序调整为短字段在前、长回复在后，并对 `style` 非空的样本做四倍重采样（样本量由 17710 增至 45985）。该数据集已就绪，相应的训练因时间限制未能完成，留作后续工作。这一干预是否有效，目前尚无实验证据。

5.2 评分口径对结论的影响

画像字段的评价依赖于评分口径，而评分脚本未公开，这一点需要在报告中明确。

若按整对象精确匹配评分，恒输出空数组可得 0.5567，高于本文模型的 0.3033，模型处于劣势。若按逐字段精确匹配评分，空输出的均值为 0.712，模型为 0.629，仍略低。但若按逐标签 micro-F1 评分，空输出的 F1 为零，模型三个字段分别为 0.2800、0.1871 与 0.0000，此时模型优于空输出。

三种口径给出的结论并不一致。由于无法确认评测方采用哪一种，我们没有对该字段做保守化后处理——若按 F1 评分，把输出压成空数组会主动放弃全部收益。这一取舍已在上文如实记录。

5.3 其他限制

模型规模是根本约束。1.5B 在情感细分上的混淆、以及画像标签的判别力不足，都与容量相关；若能在 7B 规模上训练，预期有实质改善，但这需要赛事平台提供可用算力。

序列长度 2048 使约 22% 的样本尾部被截断，长对话的后半段信息未能进入训练。

回复风格方面，训练数据本身含有 18.4% 的 markdown 加粗与 11.2% 的编号列表，模型学到该风格是对参考分布的正确拟合；但从人类阅读习惯看，这一风格偏说教，属于数据特性而非模型缺陷。

最后，镜像在本机以标准 `docker save` 格式直接组装（训练机上没有 Docker 与 WSL），其中包含 `/root` 下的推理程序、模型权重与本报告，但不包含 vLLM 基础运行时。镜像能否在官方环境启动，取决于平台是覆盖 `/root` 还是直接运行该镜像，这一点我们无法在本地验证。

6 结论与后续工作

本工作完成了从官方数据到可提交模型的完整链路：构建了 17710 条与官方提示词严格一致的监督样本，在 1.5B 规模上完成 LoRA 微调，情感标签识别达到 70.67%，约为平凡基线的 2.49 倍，格式正确率 100%。用户画像字段未达预期，其中 `style` 完全失效，我们在第四节与第五节给出了数据层面的证据与机制推断。

后续工作按优先级排列如下。首先是完成字段重排与标签重采样的对照实验，这是针对 `style` 失效的直接干预，数据已就绪。其次是在获得平台算力后迁移到 7B 规模，预期改善情感细分与画像标签。第三是为画像字段引入逐标签置信度校准，在精确匹配与 F1 两种口径之间寻找折中。最后是依据参考分布控制回复中的 markdown 与编号列表比例。

7 复现说明

数据构建、训练、合并与评测的脚本均随镜像提供。训练环境为 Python 3.12.10、torch 2.6.0+cu124、transformers 5.6.0、peft 0.18.1、LLaMA-Factory 0.9.5。评测使用官方 `run_inference.py`，评分脚本复现了官方的合法化逻辑。全部数值均来自实际运行，未做修饰。